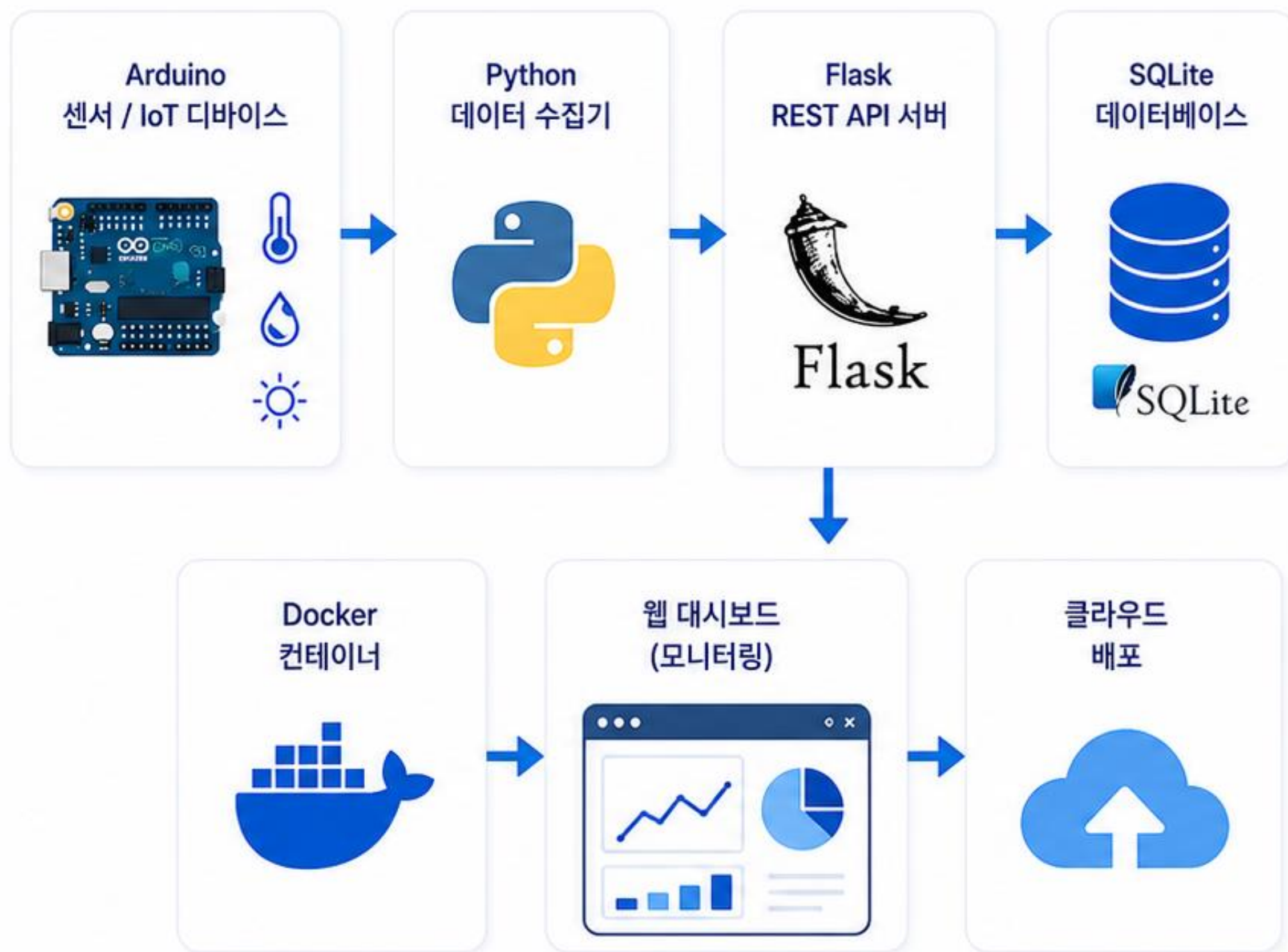


Day 5. Docker · 배포 · 최종 프로젝트

IoT 시스템을 하나의 서비스로 완성하기



Day 5.

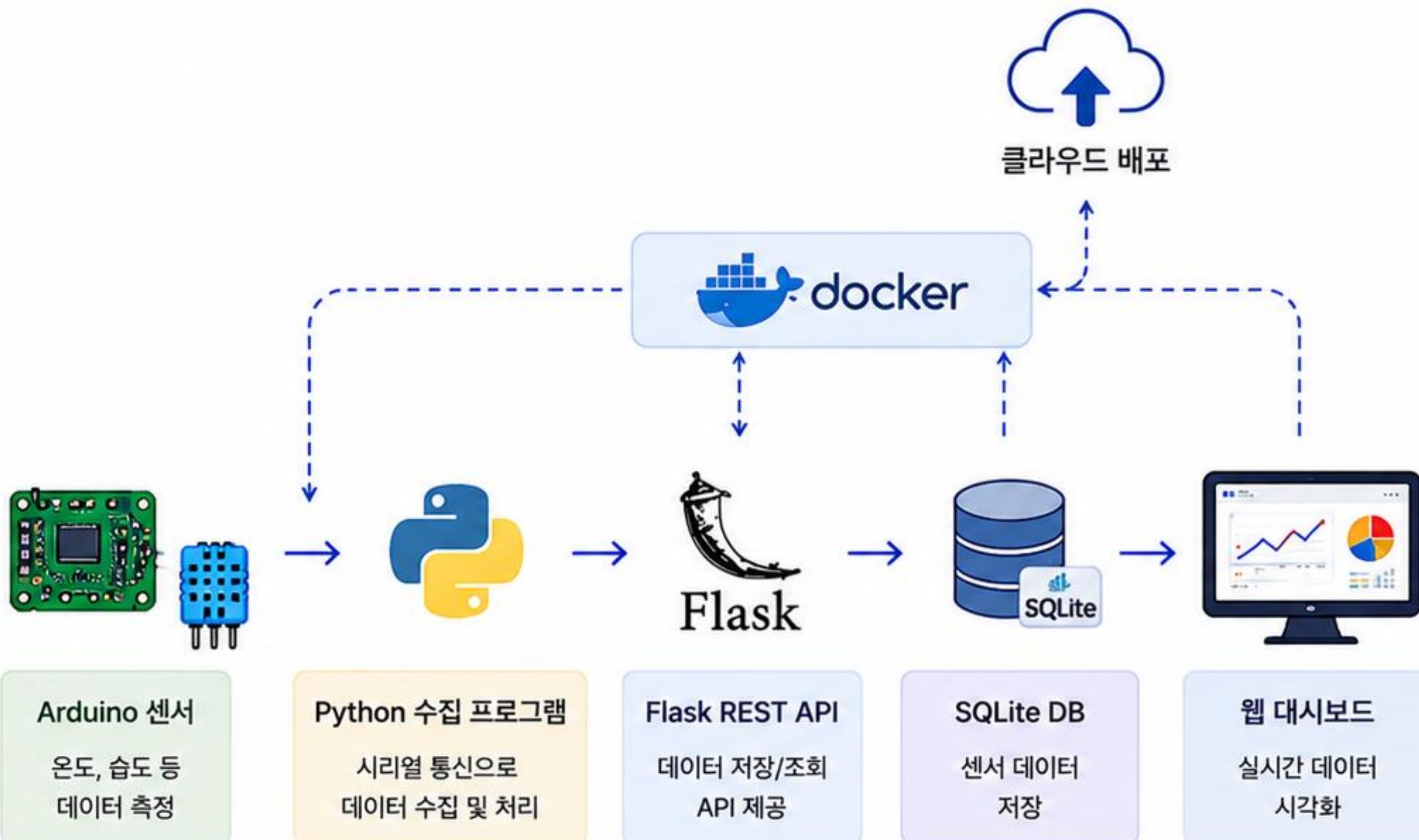
Docker · 배포 · 최종 프로젝트

IoT 시스템을 하나의 서비스로 완성하기



임베디드 · IoT 개발자 과정

Python 기반 IoT 서비스 개발



하나의 흐름으로 연결된 IoT 시스템

센서 데이터 수집 → 저장 → 분석 → 시각화 → 배포까지 전체 과정을 완성합니다.

오늘의 학습 목표

IoT 서비스를 실제 환경에서 **실행**하고 **배포**하기 위한 기술을 배우고,
최종 프로젝트를 **완성**합니다.

1



전체 IoT 시스템 구조 복습

Day 1~4에서 배운 내용을 전체 흐름으로 정리

2



Docker 개념 이해

이미지, 컨테이너, Dockerfile, Volume 이해

3



Flask 서버 컨테이너화

Flask API 서버를 Docker 이미지로 만들고 실행

4



Docker Compose로 서비스 통합

여러 컨테이너를 하나의 서비스로 관리하고 실행

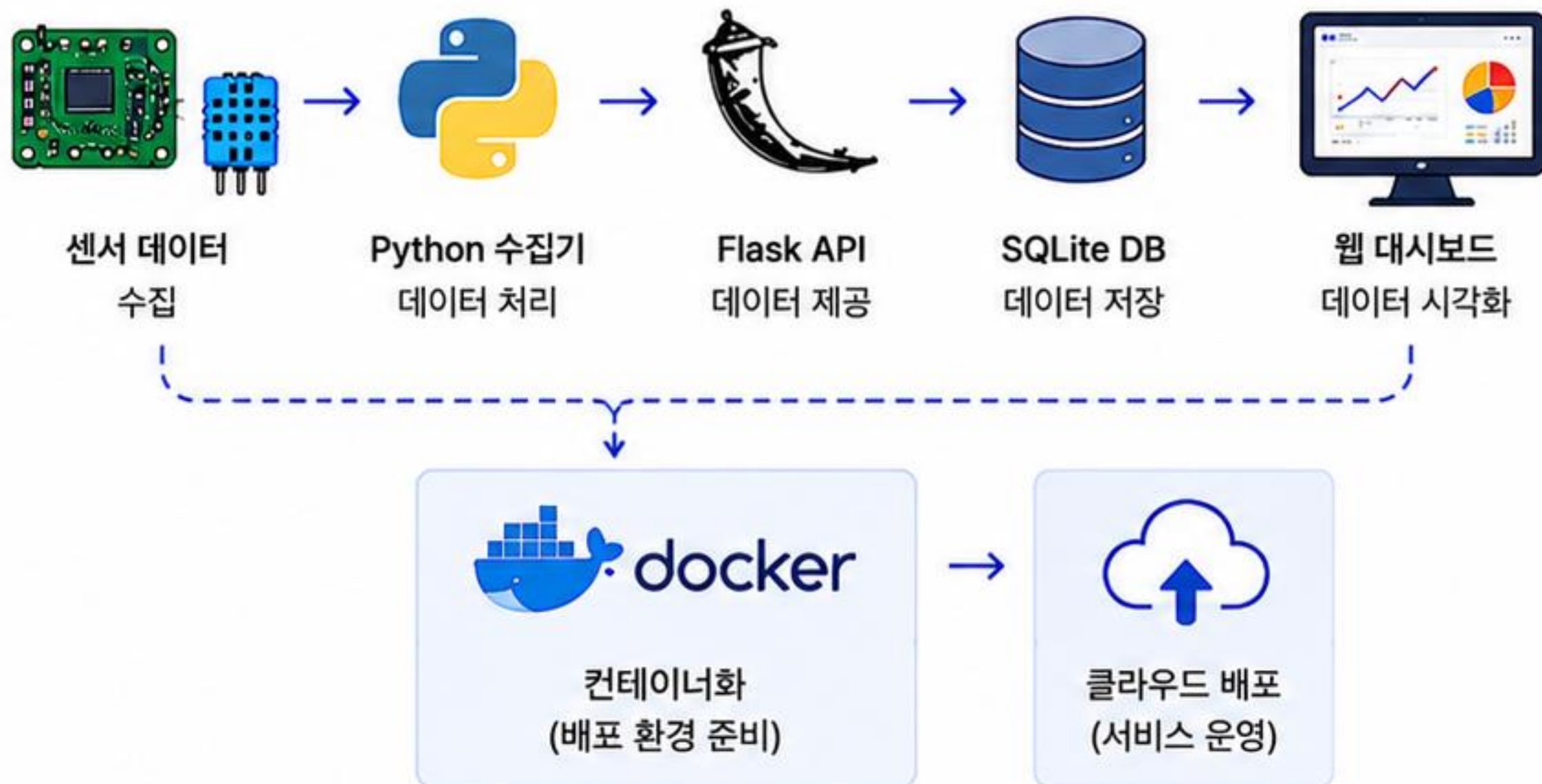
5



최종 프로젝트 완성 및 발표

전체 시스템을 완성하고 시연 및 발표 진행

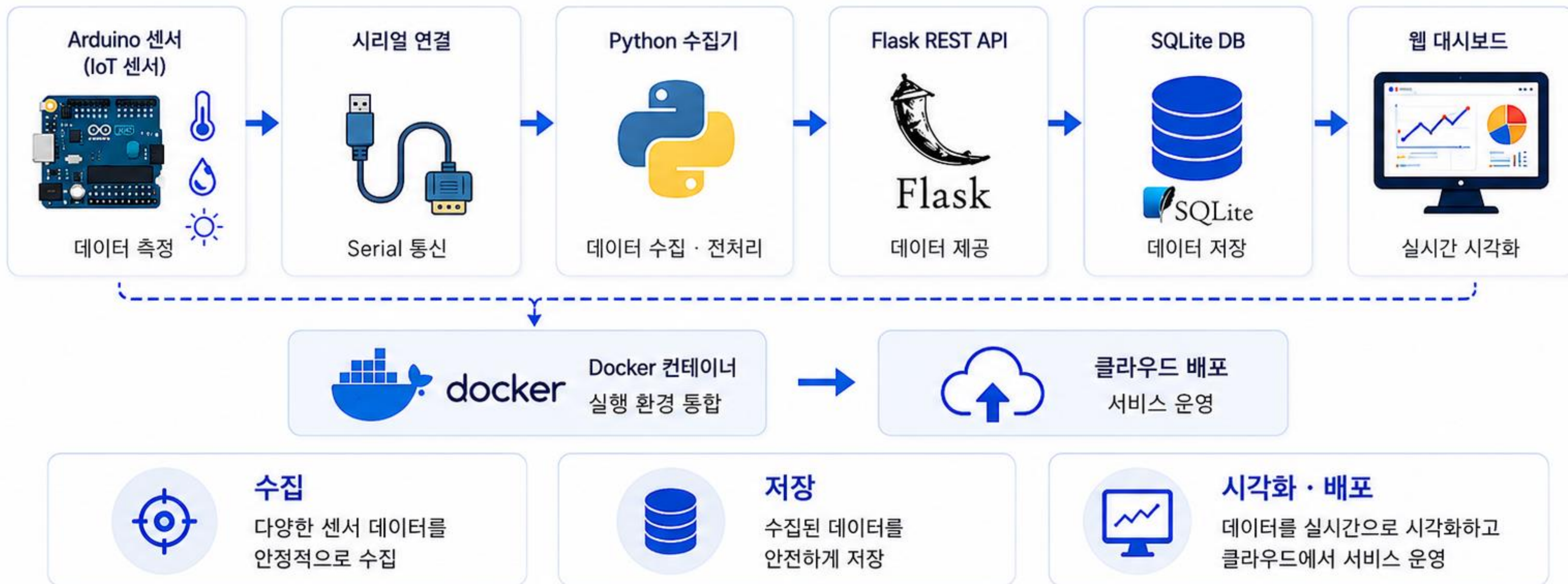
오늘의 학습 로드맵



목표

IoT 데이터를 수집부터 시각화, 배포까지 **하나의 서비스**로 완성합니다!

최종 IoT 시스템 구조



✓ Day 1~4에서 배운 내용을 하나의 서비스로 통합합니다.

왜 Docker가 필요한가?

개발 환경과 배포 환경의 차이로 생기는 문제를 해결하기 위해 Docker를 사용합니다.

❌ 문제 상황 (Docker가 없을 때)

내 PC에서는 잘 실행됨



다른 PC에서는 오류 발생



⚠️ 문제 원인

- 각 PC마다 설치된 Python 버전과 패키지가 다름
- 라이브러리 버전 차이로 실행 오류 발생
- 서버에 배포할 때 환경 설정이 복잡하고 시간이 많이 듦

✅ 해결 방법 (Docker 사용)

Docker는 실행 환경을 통째로 포장합니다!



어디서나 같은 방식으로 실행됩니다!

Docker를
사용하면?



실행 환경을 통째로 포장
필요한 모든 것을 포함하여
이동과 배포가 쉬워집니다.



어디서나 동일하게 실행
내 PC, 다른 PC, 서버, 클라우드까지
같은 환경에서 실행됩니다.



배포와 협업이 쉬워짐
개발자와 운영자 모두
편리하게 작업할 수 있습니다.



안정성과 신뢰성 향상
의존성 충돌과 설정 오류를 줄여
더 안정적인 서비스를 제공합니다.

Day 5 Docker 기본 개념

Docker는 애플리케이션과 실행 환경을 하나로 묶어 쉽게 배포하고 실행할 수 있게 도와주는 도구입니다.

1 Image (이미지) 실행 환경 설계도



컨테이너를 만들기 위한 템플릿입니다.
애플리케이션 코드, 실행 환경, 필요한
패키지와 설정이 포함되어 있습니다.

변경되지 않는 읽기 전용

2 Container (컨테이너) 실행 중인 프로그램



이미지로부터 생성된 실행 인스턴스입니다.
격리된 환경에서 애플리케이션이
실행됩니다.

실행 가능하고 격리된 환경

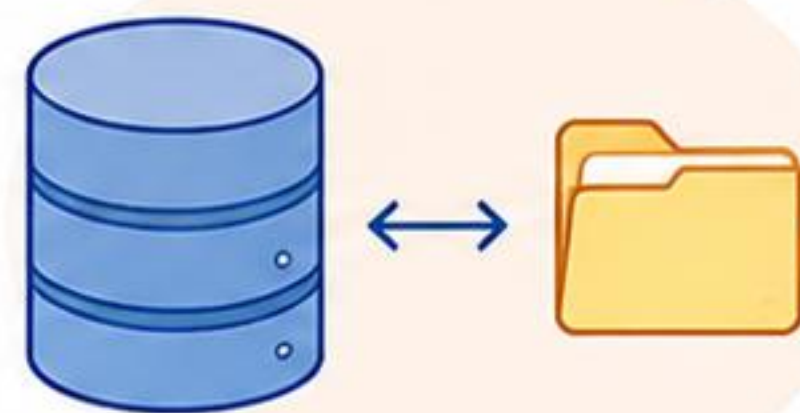
3 Dockerfile 이미지 제작 설명서

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r
requirements.txt
CMD ["python", "app.py"]
```

이미지를 만들기 위한 단계를 코드로
작성한 파일입니다.
누구나 같은 이미지로 빌드할 수 있습니다.

빌드 절차를 정의하는 파일

4 Volume (볼륨) 데이터 보관 공간



컨테이너의 데이터를 외부에 저장하여
컨테이너가 삭제되어도 데이터를
안전하게 보존할 수 있습니다.

지속 가능한 데이터 저장소



핵심 정리



이미지 생성
(Dockerfile로 빌드)



컨테이너 실행
(이미지로 실행)



데이터 보존
(Volume 사용)



어디서나 동일하게 실행
(배포 및 운영)

가상머신(VM) vs Docker

배포 방식의 차이를 이해하기

가상머신 (VM)



애플리케이션
(앱 스택)



Guest OS
(게스트 운영체제)



Hypervisor
(가상화 계층)



Host OS
(호스트 운영체제)



Hardware
(물리 하드웨어)

Guest OS 포함



구조



앱 + 필요한
환경만 포함

상대적으로 느림



실행 속도



빠름

무거움



용량



가벼움

복잡함



배포/환경 재현



쉬움

Docker



컨테이너



컨테이너



컨테이너

...



Docker Engine
(도커 엔진)



Host OS
(호스트 운영체제)



Hardware
(물리 하드웨어)



IoT 서버 배포에는 **가볍고 재현성이 높은 Docker**가 적합합니다.



Dockerfile이란?

컨테이너 이미지를 만드는 레시피



이미지 제작 설명서



실행 환경을 코드로 기록



누구나 같은 이미지 생성



배포 자동화에 유용

Dockerfile은 컨테이너 이미지를 만들기 위한 단계별 지시문입니다.

예시 Dockerfile

```

1 FROM python:3.11
2 WORKDIR /app
3 COPY . .
4 RUN pip install -r requirements.txt
5 CMD ["python", "app.py"]

```



Dockerfile



docker build



Image



Container

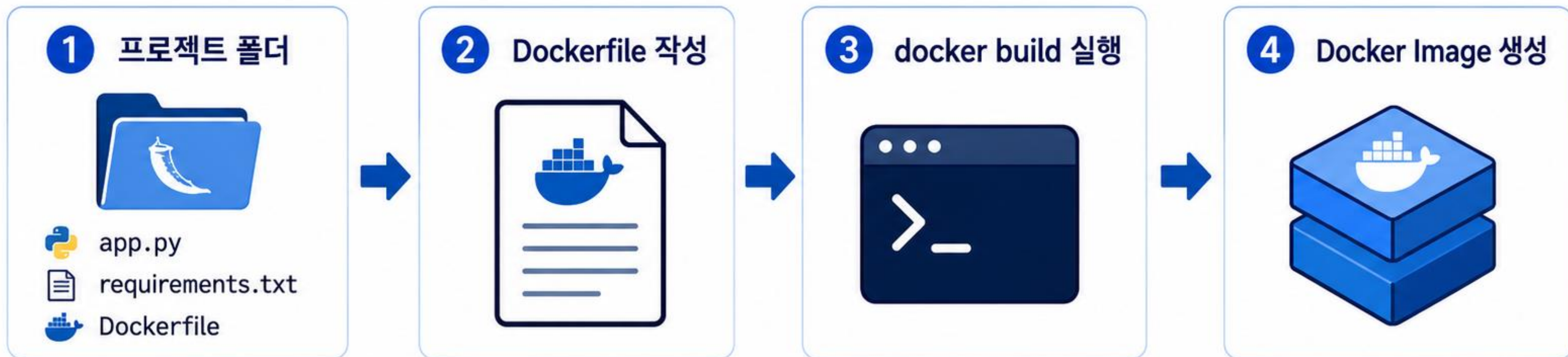


핵심: Dockerfile은 실행 환경을 코드로 남겨 같은 결과를 재현하게 해줍니다.



Flask 서버를 Docker 이미지로 만들기

프로젝트 폴더에서 실행 가능한 이미지 생성



```
docker build -t flask-iot-api .
```

- ✓ 프로젝트 파일을 기반으로 이미지 생성
- ✓ 실행 환경과 의존성을 함께 포함
- ✓ 같은 이미지를 어디서나 실행 가능



핵심: 이미지는 실행 가능한 서버 환경의 포장본입니다.



컨테이너 실행하기

Docker Image를 실행하여 Flask 서버 시작

1 Docker Image



2 Container 실행



3 브라우저 접속



```
docker run -p 5000:5000 flask-iot-api
```



접속 주소 예시

<http://localhost:5000>

- ✓ 이미지를 실행해 컨테이너 생성
- ✓ 5000 포트를 외부와 연결
- ✓ 브라우저에서 Flask API 접속



핵심: 포트 연결을 해야 외부에서 서버에 접속할 수 있습니다.

실제 Flask 웹 서버를 Docker 컨테이너로 실행하고 웹에서 확인해봅시다.



실습 목표

Flask 웹 앱을 Docker 컨테이너로 실행하고 웹 브라우저에서 확인하기



실습 단계

1



프로젝트 폴더 준비

app.py, requirements.txt 작성

2



Dockerfile 작성

Flask 앱을 위한 Docker 이미지 정의

3



이미지 빌드

docker build -t flask-app .

4



컨테이너 실행

docker run -d -p 5000:5000 --name flask-container flask-app

5



웹 브라우저 확인

http://localhost:5000 접속하여 결과 확인

Flask 앱 (app.py)

```
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route('/')
6  def home():
7      return "<h1>Hello, Docker! 🚀</h1>"
8
9  if __name__ == '__main__':
10     app.run(host='0.0.0.0', port=5000)
```

실행 명령어

```
$ docker build -t flask-app .
=> [internal] load build definition
=> [internal] load .dockerignore
=> [1/4] FROM python:3.11-slim
=> [2/4] WORKDIR /app
=> [3/4] COPY . .
=> [4/4] RUN pip install -r requirements.txt
=> exporting to image
=> writing image sha256:xxxx...
✓ Successfully built flask-app

$ docker run -d -p 5000:5000 \
  --name flask-container flask-app
c1a2b3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0

$ docker ps
CONTAINER ID   IMAGE     PORTS
c1a2b3d4e5f6   flask-app 0.0.0.0:5000->5000/tcp  flask-container
```

실행 결과 (브라우저)



TIP

컨테이너를 중지하려면

docker stop flask-container

| 다시 실행하려면

docker start flask-container

| 로그 확인은

docker logs flask-container

Flask 앱 Docker 이미지 만들기

Flask 웹 앱을 Docker 이미지로 빌드하여 배포 준비를 합니다.

Docker 이미지 빌드 과정

- 
Dockerfile 준비
 Flask 앱 실행에 필요한 환경과 명령어를 정의
- 
이미지 빌드
 docker build 명령어로 이미지 생성
- 
이미지 확인
 생성된 이미지를 로컬에서 확인
- 
(선택) 이미지 저장/공유
 다른 환경에서 사용하거나 레지스트리에 푸시

Dockerfile 예시

```

1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY app.py .
6 EXPOSE 5000
7 ENV FLASK_APP=app.py
8 ENV FLASK_RUN_HOST=0.0.0.0
9 ENV FLASK_RUN_PORT=5000
10 CMD ["python", "app.py"]
11

```

빌드 실행 예시

```

$ docker build -t flask-app .
[+] Building 12.3s (9/9) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> [internal] load .dockerignore 0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 2.1s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:... 2.3s
=> [2/5] WORKDIR /app 0.0s
=> [3/5] COPY requirements.txt . 0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 6.7s
=> [5/5] COPY app.py . 0.1s
=> exporting to image 0.2s
=> => writing image sha256:3b7e8c888... 0.0s
=> => naming to docker.io/library/flask-app:latest 0.0s

```

빌드 명령어

```
$ docker build -t flask-app .
```



이미지 확인

```

$ docker images

```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
flask-app	latest	3b7e8c2a9f1d	2 minutes ago	123MB
python	3.11-slim	e4a1c2d3f4b5	3 days ago	121MB
...				



TIP

- -t 옵션으로 이미지 이름과 태그를 지정합니다.
- . (점)은 현재 폴더를 빌드 컨텍스트로 지정합니다.
- 이미지가 생성되면 컨테이너로 실행할 준비 완료!

Docker 컨테이너 실행과 관리

이미지로 컨테이너를 실행하고, 실행 중인 컨테이너를 관리하는 기본 명령어를 알아봅시다.

1. 컨테이너 실행 (run)

```
$ docker run -d -p 5000:5000 --name flask-container flask-app
```



이미지
(flask-app)



docker run



실행 중인 컨테이너
(flask-container)

주요 옵션

- -d : 백그라운드 실행 (detached)
- -p : 포트 매핑 (호스트:컨테이너)
- --name : 컨테이너 이름 지정

2. 실행 중인 컨테이너 확인

```
$ docker ps
```

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
c1a2b3d4e5f6	flask-app	"python app.py"	Up 2 minutes	0.0.0.0:5000->5000/tcp	flask-container

3. 모든 컨테이너 확인 (실행 중 + 중지됨)

```
$ docker ps -a
```

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
c1a2b3d4e5f6	flask-app	"python app.py"	Up 2 minutes	0.0.0.0:5000->5000/tcp	flask-container
a1b2c3d4e5f6	flask-app	"python app.py"	Exited (0) 1 hour ago	<none>	flask-old

4. 컨테이너 중지, 시작, 재시작



중지 (stop)

```
$ docker stop flask-container
```

실행 중인 컨테이너를
안전하게 중지합니다.



시작 (start)

```
$ docker start flask-container
```

중지된 컨테이너를
다시 시작합니다.



재시작 (restart)

```
$ docker restart flask-container
```

실행 중인 컨테이너를
다시 시작합니다.

5. 컨테이너 삭제

```
$ docker stop flask-container # 먼저 중지 (실행 중인 경우)
$ docker rm flask-container # 컨테이너 삭제
```

⚠ 주의 사항

실행 중인 컨테이너는
바로 삭제할 수 없습니다.
먼저 중지한 후 삭제하세요.



핵심 정리



이미지 선택
(flask-app)



컨테이너 실행
(docker run)



컨테이너 관리
(ps, stop, start 등)



컨테이너 삭제
(rm)



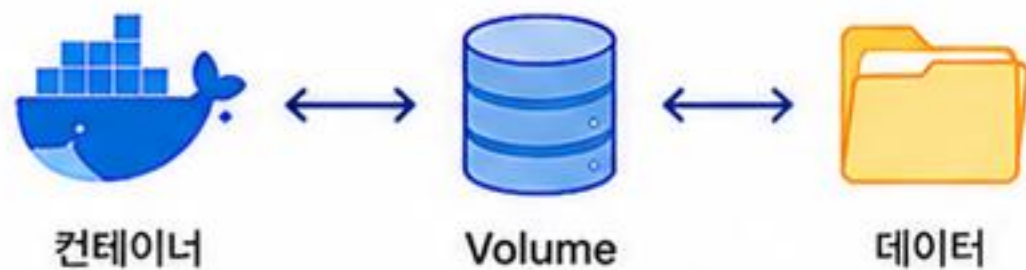
기억할 포인트

이미지로부터 컨테이너를 만들고,
필요할 때 관리하고 정리합니다!

Volume을 사용하면 컨테이너가 삭제되어도 데이터를 안전하게 보존할 수 있습니다.

Volume이란?

컨테이너 외부의 저장 공간으로, 데이터를 영구적으로 보관할 수 있게 해줍니다.
컨테이너가 삭제되어도 Volume의 데이터는 유지됩니다.



Volume의 장점

- ✓ 데이터 영구 보존
- ✓ 여러 컨테이너에서 데이터 공유
- ✓ 백업 및 복원 용이
- ✓ 환경 분리 및 관리 편리

>_- 실습: Flask 앱 데이터를 Volume에 저장하기

1 Volume 생성

```
$ docker volume create flask-data
```

2 컨테이너 실행 (Volume 연결)

```
$ docker run -d -p 5000:5000 \
  --name flask-app \
  -v flask-data:/app/data \
  flask-app
```

3 컨테이너 내부에 데이터 생성

```
$ docker exec flask-app sh -c "echo 'Hello Volume!' > /app/data/hello.txt"
```

4 컨테이너 삭제

```
$ docker rm -f flask-app
```

5 새 컨테이너로 다시 실행 (동일 Volume 사용)

```
$ docker run -d -p 5000:5000 --name flask-app -v flask-data:/app/data flask-app
```

6 데이터 확인

```
$ docker exec flask-app cat /app/data/hello.txt # Hello Volume!
```

실습 구조



>_- Volume 관련 명령어

<code>docker volume ls</code>	생성된 Volume 목록 확인
<code>docker volume create <이름></code>	Volume 생성
<code>docker volume inspect <이름></code>	Volume 상세 정보 확인
<code>docker volume rm <이름></code>	Volume 삭제 (사용 중이면 삭제 불가)
<code>docker volume prune</code>	사용하지 않는 Volume 삭제



TIP



데이터 보호

DB, 업로드 파일, 로그 등 중요한 데이터는 반드시 Volume을 사용하여 보관하세요.



백업

Volume 데이터는 호스트의 docker 저장 경로에서 백업할 수 있습니다.

⚠ 주의 사항

- Volume을 삭제하면 데이터도 함께 삭제됩니다.
- 컨테이너 내부 경로와 Volume 경로를 정확히 매핑해야 합니다.

Docker Volume 실습 예제

Flask 앱 데이터를 Volume에 저장하여 컨테이너를 삭제해도 데이터가 유지되는 것을 확인해 봅시다.

1. 실습 목표

Flask 앱이 생성한 데이터를 Volume에 저장하고,
컨테이너를 삭제/재생성해도 데이터가 유지되는지 확인

2. Docker Compose 파일 (docker-compose.yml)

```
1 version: '3.8'
2 services:
3   web:
4     build: .
5     ports:
6       - "5000:5000"
7     volumes:
8       - flask-data:/app/data
9     environment:
10      - FLASK_APP=app.py
11      - FLASK_RUN_HOST=0.0.0.0
12 volumes:
13   flask-data:
```

 flask-data Volume

컨테이너 외부에 생성되어
데이터를 안전하게 보관

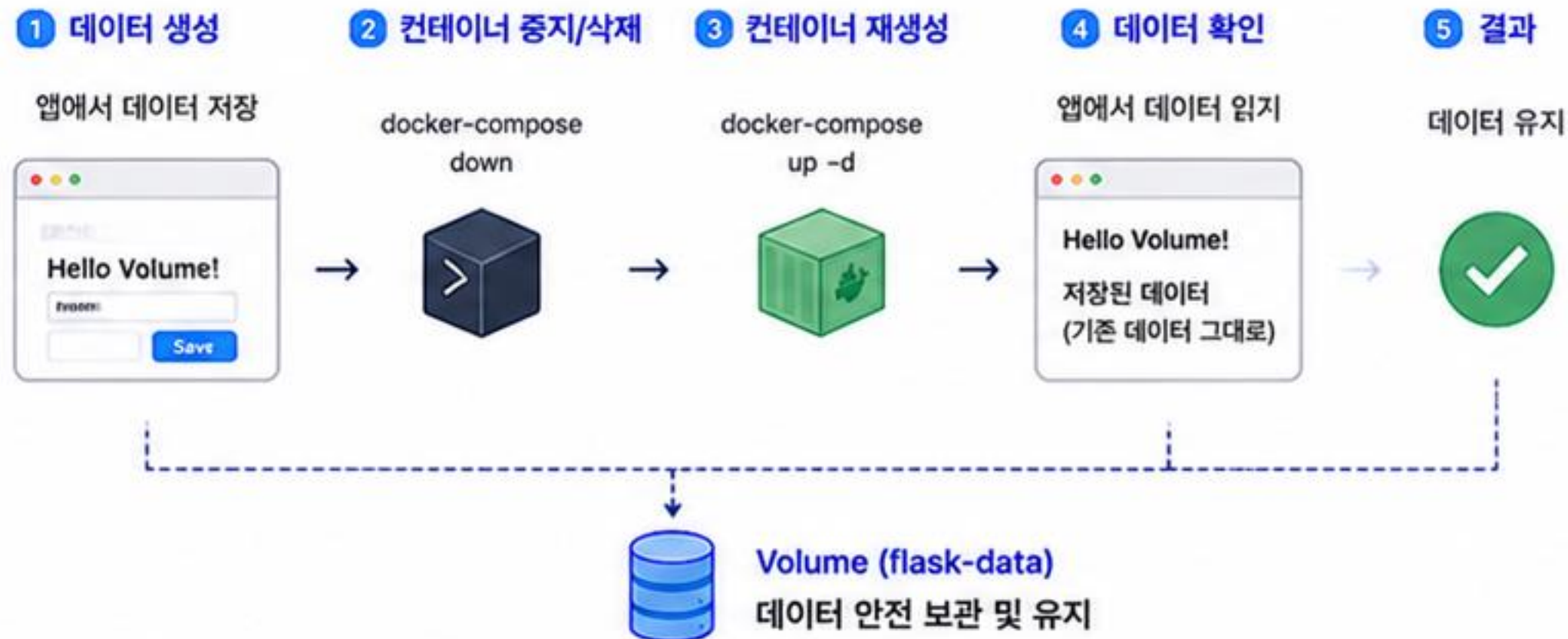
3. 실행 및 사용

- 1 docker-compose up -d --build
- 2 브라우저에서 http://localhost:5000 접속
- 3 데이터를 입력하고 /app/data/ 에 저장 확인

4. 동작 확인

- ✓ 데이터를 추가하면 /app/data/에 파일 생성
- ✓ 컨테이너를 삭제해도 Volume에 데이터 유지
- ✓ 컨테이너를 다시 실행해도 데이터 그대로 복제

5. 컨테이너 재생성 후 데이터 유지 확인 과정



6. 확인 명령어

```
# Volume 목록 확인
docker volume ls

# Volume 상세 정보 확인
docker volume inspect flask-data

# Volume 내부 데이터 확인
docker run --rm -v flask-data:/data alpine ls /data
```

7. 정리

```
docker-compose down -v
# Volume까지 모두 삭제
```

⚠ 주의

-v 옵션을 사용하면 Volume의 데이터도 함께 삭제됩니다.
실습이 끝난 후에만 사용하세요.



TIP

개발 환경에서는 코드 변경이 잦으므로,
코드는 바인드 마운트(Volume 아님)로, 데이터는 Volume으로 관리하는 것이 좋습니다!



코드 (변경 빈번)



바인드 마운트

(Host ↔ Container)



데이터 (영구 보관)



Volume

(Container 외부 저장)

Day 5 Docker Volume 실습 예제 (결과 확인)

Flask 앱 데이터를 Volume에 저장했으므로, 컨테이너를 삭제해도 데이터가 유지되는 것을 확인해 봅시다.

1. 컨테이너 실행 및 데이터 생성

```
$ docker-compose up -d --build
$ curl http://localhost:5000
```



✅ 데이터 파일 확인

```
$ docker exec flask-app
cat /app/data/hello.txt
```

Hello Volume!

2. 컨테이너 및 앱 삭제

```
$ docker-compose down -v
# 컨테이너, 네트워크 삭제 (Volume은 유지)
```

⚠️ 주의

-v 옵션은 Volume까지 삭제합니다.
실습에서는 -v 없이 실행합니다!

3. Volume 존재 확인

```
$ docker volume ls
```

DRIVER	VOLUME NAME
local	flask-data

ℹ️ Volume은 아직 존재!

Volume에 저장된 데이터는
삭제되지 않았습니다.

4. 컨테이너 재실행

```
$ docker-compose up -d
$ curl http://localhost:5000
```



✅ 데이터 유지 확인

```
$ docker exec flask-app
cat /app/data/hello.txt
```

Hello Volume!

5. 전체 확인 과정 요약



💡 핵심 정리

- ✓ Volume을 사용하면 컨테이너를 삭제해도 데이터가 안전하게 보존됩니다.
- ✓ 상태가 필요한 데이터(DB, 파일 등)는 반드시 Volume에 저장해야 합니다.

⚠️ Volume 주의사항

- Volume을 삭제하려면 아래 명령을 사용합니다.

```
$ docker volume rm flask-data
```
- Volume을 사용하지 않으면 컨테이너 삭제 시 데이터도 함께 사라집니다.

실습 구조 (최종 상태)



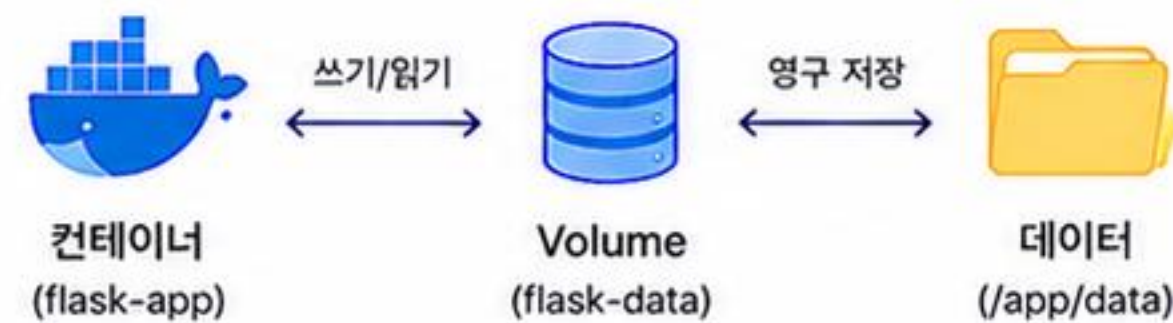
관련 명령어 정리

<code>docker volume create <이름></code>	Volume 생성
<code>docker volume ls</code>	Volume 목록 확인
<code>docker volume inspect <이름></code>	Volume 상세 정보 확인
<code>docker volume rm <이름></code>	Volume 삭제
<code>docker run -v <이름>:<경로> ...</code>	Volume을 컨테이너에 연결

Docker Volume을 사용하여 Flask 앱 데이터를 안전하게 관리하는 방법을 정리합니다.

1. 실습 개요

Flask 앱의 데이터를 Volume에 저장하여
컨테이너를 삭제/재생성해도 데이터가 유지됨을 확인!했습니다.



2. 주요 명령어

<code>docker volume create <이름></code>	Volume 생성
<code>docker volume ls</code>	Volume 목록 확인
<code>docker run -v <이름>:<경로> ...</code>	컨테이너에 Volume 연결
<code>docker run --rm -v <이름>:<경로> ...</code>	일회성 컨테이너 실행
<code>docker volume inspect <이름></code>	Volume 상세 정보 확인
<code>docker volume rm <이름></code>	Volume 삭제

3. 실습 흐름 요약



4. 결과 확인

1차 실행 후 데이터 확인

`$ docker exec flask-app cat /app/data/hello.txt`

Hello Volume!

데이터 생성됨

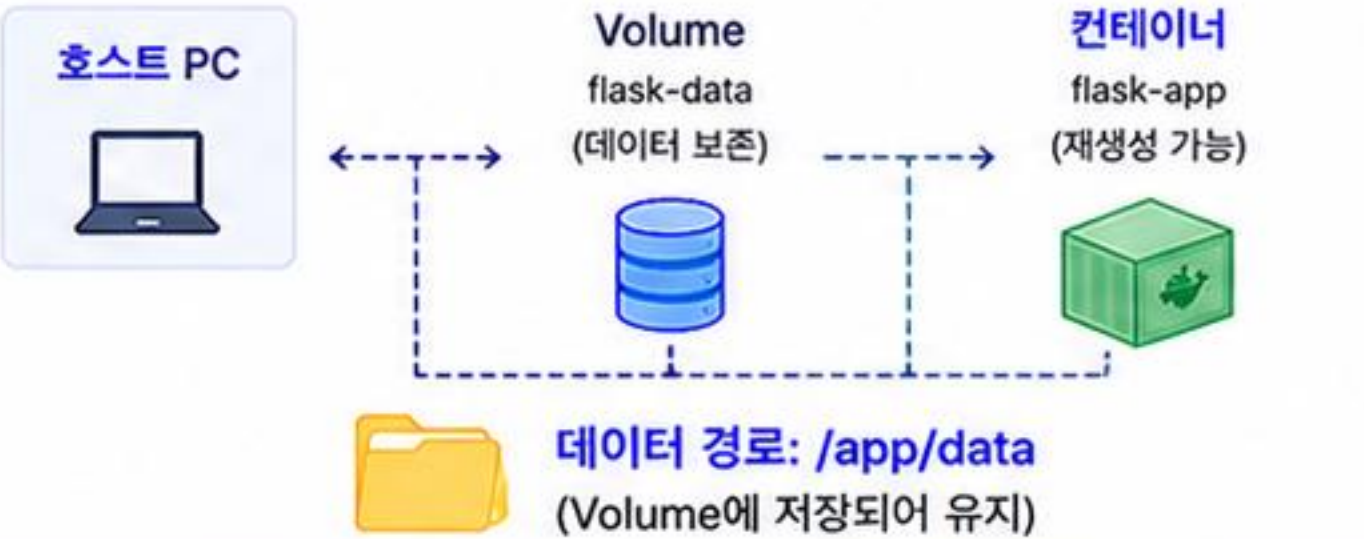
컨테이너 재생성 후 데이터 확인

`$ docker exec flask-app cat /app/data/hello.txt`

Hello Volume!

데이터 유지됨!

5. 실습 구조 (최종 상태)



💡

6. 핵심 포인트

- ✓ Volume은 컨테이너 외부에 데이터를 보관하여 안전하게 유지합니다.
- ✓ 컨테이너가 삭제되어도 Volume에 저장된 데이터는 그대로 남아 있습니다.

- ✓ 테스트, 개발, 운영 환경에서 데이터 손실을 방지할 수 있습니다.
- ✓ 바인드 마운트와 달리 호스트 경로에 의존하지 않아 이식성이 좋습니다.

💡

TIP

운영 환경에서는 DB, 업로드 파일, 로그 등 중요한 데이터는 반드시 Volume을 사용하여 관리하세요!

지금까지 배운 내용을 바탕으로 Docker Volume을 사용하여 데이터를 안전하게 관리하는 실습 과제입니다.

🎯. 실습 목표

- ✓ Flask 앱 데이터를 Volume에 저장
- ✓ 컨테이너 삭제 후에도 데이터 유지 확인
- ✓ Volume 관련 명령어를 사용하여 관리

📄. 요구 사항

- 1 Flask 앱을 사용합니다.
- 2 /app/data 경로에 데이터를 저장합니다.
- 3 Named Volume을 사용합니다.
- 4 컨테이너를 삭제하고 재생성해도 데이터가 유지되어야 합니다.

3.. 실습 단계

 Step 1	Volume 생성
 Step 2	Volume을 사용하여 컨테이너 실행
 Step 3	컨테이너 내부에 데이터 생성
 Step 4	컨테이너 중지 및 삭제
 Step 5	새 컨테이너로 다시 실행
 Step 6	데이터 유지 확인

4. 실습 명령어 예시

- Step 1

Named Volume 생성

```
$ docker volume create flask-data
```
- Step 2

Volume을 사용하여 컨테이너 실행

```
$ docker run -d -p 5000:5000 \
  --name flask-app \
  -v flask-data:/app/data \
  flask-app
```
- Step 3

컨테이너 내부에 데이터 생성

```
$ docker exec flask-app sh -c 'echo "Hello Volume!" > /app/data/hello.txt'
```
- Step 4

컨테이너 중지 및 삭제

```
$ docker stop flask-app
$ docker rm flask-app
```
- Step 5

새 컨테이너로 다시 실행

```
$ docker run -d -p 5000:5000 \
  --name flask-app \
  -v flask-data:/app/data \
  flask-app
```
- Step 6

데이터 유지 확인

```
$ docker exec flask-app cat /app/data/hello.txt
Hello Volume!
```

⚠ 주의사항

- Volume을 삭제하면 해당 데이터도 함께 사라집니다.
- 실습이 끝난 후에는 불필요한 Volume을 정리해 주세요.

✅ 확인 사항

- ✓ 컨테이너를 삭제해도 /app/data/hello.txt 파일이 유지됨
- ✓ 새 컨테이너에서 동일한 파일 내용을 확인할 수 있음
- ✓ docker volume ls 명령으로 Volume 존재 확인

6. 추가 명령어 (참고)

<code>docker volume ls</code>	Volume 목록 확인
<code>docker volume inspect <이름></code>	Volume 상세 정보 확인
<code>dacker volume rm <이름></code>	Volume 삭제 (사용 중이면 삭제 불가)
<code>docker volume prune</code>	사용하지 않는 Volume 일괄 삭제

💡 Tip

Volume은 컨테이너와 독립적으로 존재하므로, 데이터 백업, 공유, 영구 저장이 가능합니다.



🏆 도전 과제

- ★ 여러 컨테이너에서 같은 Volume을 공유해 보세요.
- ★ 다른 경로(/app/log 등)에도 Volume을 마운트해 보세요.

Day 5 Docker Volume 실습: 데이터 변경 확인

컨테이너를 재시작하거나 삭제해도 Volume에 저장된 데이터가 그대로 유지되는 것을 확인해 봅시다.

1. 실습 시나리오

Flask 앱이 /app/data/hello.txt 파일에 데이터를 쓰고, 컨테이너를 재시작/삭제해도 데이터가 유지되는지 확인합니다.



2. 단계별 실습

Step 1	컨테이너 실행 및 파일 생성
Step 2	생성된 데이터 확인
Step 3	컨테이너 중지 후 재시작
Step 4	데이터 유지 확인
Step 5	컨테이너 삭제 후 재생성
Step 6	데이터 다시 확인

3. 실습 명령어 모음

```
# 1. 컨테이너 실행
docker run -d -p 5000:5000 --name flask-app -v flask-data:/app/data flask-app

# 2. 데이터 생성
docker exec flask-app sh -c "echo 'Hello Volume!' > /app/data/hello.txt"

# 3. 컨테이너 중지
docker stop flask-app

# 4. 컨테이너 재시작
docker start flask-app

# 5. 컨테이너 삭제
docker rm -f flask-app

# 6. 새 컨테이너로 재실행
docker run -d -p 5000:5000 --name flask-app -v flask-data:/app/data flask-app
```

4. 단계별 결과 확인

- 1 컨테이너 실행 및 파일 생성
`echo 'Hello Volume!' > /app/data/hello.txt` ✓ 파일 생성 완료
- 2 생성된 데이터 확인
`cat /app/data/hello.txt`
Hello Volume! ✓ 데이터 확인
- 3 컨테이너 중지 후 재시작
`docker stop flask-app && docker start flask-app` ✓ 재시작 완료
- 4 데이터 유지 확인
`cat /app/data/hello.txt`
Hello Volume! ✓ 데이터 유지!
- 5 컨테이너 삭제 후 재생성
`docker rm -f flask-app`
`docker run -d -p 5000:5000 --name flask-app -v flask-data:/app/data flask-app` ✓ 새 컨테이너 실행
- 6 데이터 다시 확인
`cat /app/data/hello.txt`
Hello Volume! ✓ 데이터 그대로!

핵심 포인트

- ✓ Volume은 컨테이너의 생명주기와 독립적입니다.
- ✓ 컨테이너가 삭제되어도 Volume의 데이터는 남아있습니다.
- ✓ -v 옵션으로 Volume을 연결하면 데이터가 안전하게 보존됩니다.

주의사항

- Volume을 삭제하려면 별도로 `docker volume rm` 명령을 사용해야 합니다.
- 테스트 환경에서는 불필요한 Volume을 정리해 주세요.

5. 최종 결과

컨테이너를 중지, 삭제, 재생성해도 Volume(flask-data)에 저장된 데이터는 안전하게 유지되었습니다!

Volume은 컨테이너의 생명주기와 독립적으로 데이터를 보존합니다.



6. 실습 구조 (최종 상태)



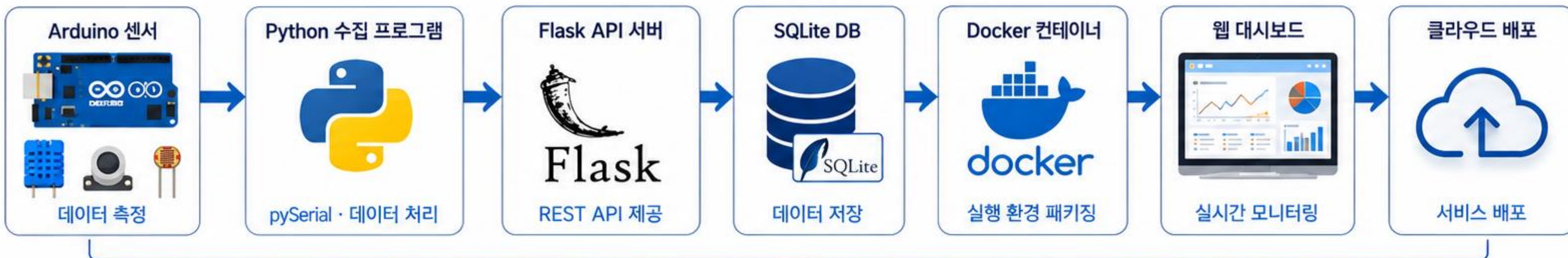
정리 명령어 (실습 후)

```
docker stop flask-app
docker rm flask-app
docker volume ls
docker volume rm flask-data
```



수고하셨습니다!

이제 여러분은 Python 기반
IoT 서비스를 직접 만들 수 있습니다.



데이터 수집
센서 데이터를 안정적으로
수집하고 처리한다



API 서버
REST API를 통해
데이터를 제공한다



웹 대시보드
실시간으로 데이터를
시각화하여 보여준다



Docker 실행
일관된 환경에서
어디서나 실행한다

 “작은 센서 데이터가 **하나의 서비스**가 됩니다.”